

## A APPENDIX

Table 1 lists the selected papers and maps each of them to the considered non-functional requirements, i.e., Privacy, Security, and Transparency. In the following, we present the complete description of the identified design objectives (DESO) and design techniques (DEST).

Table 1: Ethics-based concerns mapped to the harvested relevant papers.

| Ethics-based Weaknesses | Relevant References                        |
|-------------------------|--------------------------------------------|
| Privacy                 | [3,7,8,14,16,20,29]                        |
| Security                | [1,3,7,8,13,14,16,18,19,20,26,29,30,31,33] |
| Transparency            | [7,16,28]                                  |

### A.1 Description of Design Objectives and Techniques for Privacy

*Privacy* is a foundational ethical principle that safeguards individual rights and ensures sensitive information is handled respectfully [11]. In current CPS deployments, continuous streams of sensor data are collected and processed, intensifying privacy challenges. Therefore, users must be empowered to control and influence how their personal data is used. Overall, for the Privacy requirement, we identified 7 design objectives and 12 corresponding design techniques. We structure them into the knowledge graph reported in Figure 1.

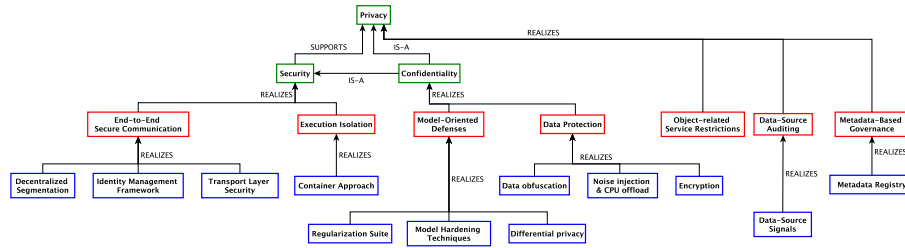


Fig. 1: Knowledge Graph mapping *Privacy* to its design objectives and their corresponding design techniques.

*Security* supports Privacy by preventing unauthorized access and restricting data handling to legitimate parties, thereby allowing individuals to maintain control of their personal information [27]. *Confidentiality* promotes Privacy by guaranteeing that sensitive data is accessible only to authorized entities.

**DESO** *End-to-End Secure Communication*. It aims to ensure that data is preserved across all layers and intermediaries, regardless of how many components are involved.

**DEST** *Decentralized Segmentation*. It uses blockchain and Software-Defined Networking (SDN) to create secure, private communication domains across fog and cloud. Blockchain ensures immutable records for node authentication and data integrity with certificate-based encryption [7]. SDN enables segmentation and centralized control, enforcing privacy by isolating sensitive traffic, applying user access rules, and dynamically adapting network slices in real-time [7].

**DEST** *Identity Management Framework*. It ensures identity and access management across all components and fog nodes of the net [7]. The framework issues and validates credentials, and it enforces access policies, ensuring that only authorized users and services can interact with the system resources. In smart city middleware, identity management is also reflected in authentication, authorization, secure access control, and security policy management for users, devices, and services [16].

**DEST** *Transport Layer Security (TLS)*. It is a protocol that guarantees confidentiality, integrity, and authenticity of all communications between the components [7,8]. By establishing an encrypted and authenticated channel, it prevents the leakage and tampering of data during exchange. In smart city middleware, this technique can also appear as secure messaging protocols such as secure MQTT with TLS, where TLS protects data exchanged between distributed middleware components, IoT devices, and smart city services [16].

**DESO** *Execution Isolation*. It aims to guarantee that each execution environment remains isolated, preventing unauthorized access, interference, or resource contention between users and system components. This objective can be supported by trusted execution environments, where critical network handling or user interaction is moved outside the control of compromised normal-world software [20,33].

**DEST** *Container Approach*. It encapsulates an application and its dependencies into a self-contained and isolated environment [7,8].

**DESO** *Model-Oriented Defenses*. They encompass methods designed to fortify CPS machine-learning models against external manipulation, while maintaining their predictive accuracy and robustness. These methods intervene to mitigate risks such as poisoning, extraction, and integrity breaches.

**DEST** *Regularization Suite*. It refers to a set of techniques such as strong l2 regularization [5], dropout, and model stacking designed to limit model capacity and promote smoother decision boundaries [3]. These methods reduce the risk of overfitting and enhance robustness, making it more difficult for attackers to perform inference or poisoning attacks.

**DEST** *Model Hardening Techniques*. They safeguard model availability and confidentiality against poisoning and model-stealing attacks [3]. Juuti et al. [21] propose the so-called PRADA (Protecting against Deep Neural Network Model Stealing Attacks) technique that monitors the distribution of sequential API queries and triggers an alert whenever it deviates from normal behavior. Ro-

bust aggregation schemes, such as Krum and coordinate-wise median, limit the influence of malicious client updates on the global model [2]. Meanwhile, ERR (Error Rate based Rejection), LFR (Loss Function based Rejection), and Union (ERR+LFR) defenses filter out local models whose updates would degrade overall performance, countering local poisoning [12]. The fine-pruning technique removes inactive, backdoor-related neurons and then fine-tunes the pruned model on benign data to restore accuracy while eliminating backdoor behavior [24].

**DEST Differential Privacy.** During training, this technique adds random noise to the data. This limits the influence of any individual record, which helps prevent attackers from inferring personal information, while still preserving the overall accuracy of the model [9].

**DESO Data Protection.** It focuses on preserving both data and model confidentiality by countering system-oriented attacks on sensitive information. This design objective encompasses the measures that prevent unauthorized access, use, disclosure, disruption, modification, or destruction of sensitive data.

**DEST Data Obfuscation.** It allows to transform data, making the original content unreadable or less informative to attackers while remaining functional for authorized operations [10].

**DEST Noise injection & CPU offload.** They disguise sensitive information and minimize exposure during processing. Noise injection adds random perturbations to obscure user data while preserving overall utility. Conversely, CPU offload transfers heavy tasks to specialized hardware, reducing CPU exposure [38].

**DEST Encryption.** It converts readable data into an unreadable format (i.e., ciphertext) using cryptographic keys [38,8,16]. A key can only revert the ciphertext to its original form, ensuring that sensitive information remains confidential.

**DESO Object-related Service Restrictions.** They enforce detailed regulations on CPS devices by limiting their operations (i.e., data exchanges) to effectively protect user privacy [14]. They prevent unintended disclosure of personal or contextual information by prohibiting unauthorized queries or actions, while maintaining consistency between physical entities and their digital counterparts.

**DESO Metadata Governance.** It aims to govern privacy- and security-relevant information through explicit metadata descriptions. In data-intensive CPS, metadata can describe the nature, semantics, quality, volume, and velocity of data, as well as monitoring metrics and security requirements such as authentication, integrity, confidentiality, and availability. This design objective realizes Privacy by making data collection, processing, and protection conditions explicit and manageable. It also realizes Security when the same metadata is used to specify and manage security requirements such as authentication, integrity, confidentiality, and availability [8].

**DEST Metadata Registry.** It provides a structured registry where metadata about data streams, processing requirements, quality properties, monitoring metrics, and security constraints is stored and managed. For Privacy, it supports the control of privacy-sensitive data processing according to registered metadata. For Security, it makes security constraints explicit and supports security-aware platform configuration, deployment, and reconfiguration [8].

**DESO Data-Source Auditing.** It aims to assess which data sources, communication channels, or external services are used by an Android application. This design objective realizes Privacy by helping auditors determine whether the application accesses only the data sources that are justified for its intended purpose. It is especially relevant for controls such as restricted data flow, data minimization, and data confidentiality, where the number and type of accessed data sources can indicate potential privacy risks. It can also support Security when the identified data sources reveal communication channels or external dependencies relevant to security assessment [29].

**DEST Data-Source Signals.** They are implementation-level indicators used to identify possible data sources accessed by an Android application. For Privacy, these signals help auditors determine whether the number and type of data sources are compatible with restricted data flow and data minimization. For Security, they help auditors assess whether the application accesses communication channels or external services that may affect controls [29].

## A.2 Description of Design Objectives and Techniques for Security

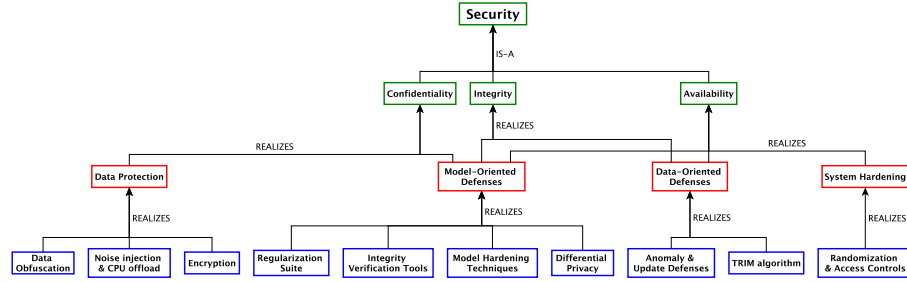


Fig. 2: Knowledge Graph mapping *Security* to its design objectives and their corresponding design techniques (Part I).

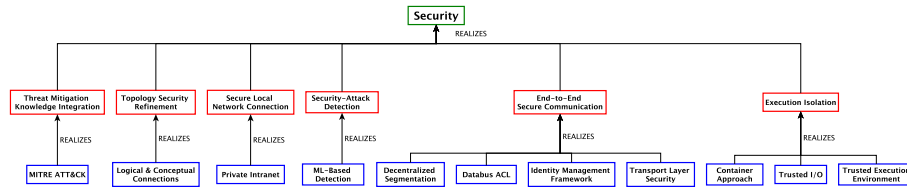


Fig. 3: Knowledge Graph mapping *Security* to its design objectives and their corresponding design techniques (Part II).

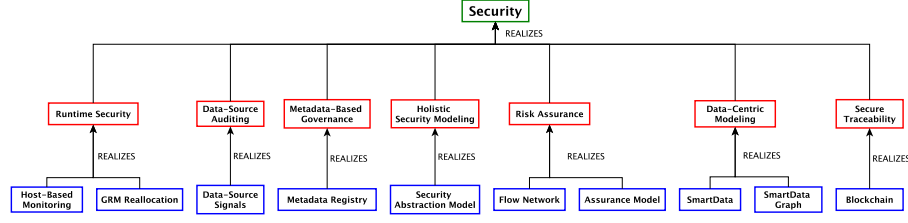


Fig. 4: Knowledge Graph mapping *Security* to its design objectives and their corresponding design techniques (Part III).

Cyber-Physical Systems show the peculiar characteristic of integrating different components, such as sensors, actuators, and control algorithms. This contributes to vulnerability, since there might be attacks that can manipulate or disrupt critical operations in the absence of robust security measures. To address these risks, we identify and organize 17 design objectives and 31 design techniques in the knowledge graph depicted in Figures 2, 3, and 4.

*Confidentiality*, *Integrity*, and *Availability* are foundational NFRs in system security [3,1]. Confidentiality prevents the unauthorized disclosure of sensitive data; integrity maintains the completeness and trustworthiness of the data throughout its lifecycle; and availability ensures that services remain online and accessible as intended.

**DEST Integrity Verification Tools** (under **DESO Model-Oriented Defenses**, see Section A.1). They ensure that a model remains untampered before the deployment and during operation. SHA hashing generates a compact representation of the model parameters and certificates, enabling the detection of unauthorized changes [34]. Input anomaly detection blocks suspicious data, while re-training on clean inputs removes inserted backdoors with minimal loss in accuracy; and autoencoder-based preprocessing helps remove illegitimate inputs before they can reach the model [37]. DeepInspect performs black-box analysis by learning the distribution of potential triggers from model queries to reveal hidden backdoor footprints [4]. TABOR approach targets varied trigger sizes, shapes, and locations and uses optimization-based regularization to improve detection accuracy [17,3].

**DESO Data-Oriented Defenses**. They aim to protect model integrity and availability by detecting and mitigating malicious alterations to training data or model updates.

**DEST Anomaly & Update Defenses**. These approaches include many techniques to prevent poisoning and backdoor attacks on training data [3]. Federated learning can address these problems by adopting specific architectural patterns (i.e., secure aggregator) that detect unusual updates from clients [25]. Stochastic Gradient Descent leverages standard gradient updates that inherently mitigate the influence of poisoned data [35]. Unsupervised Anomaly Detection learns the normal behavior of the system and flags unusual patterns, whether in network traffic,

user actions, or logs, without requiring any pre-labeled data [36]. Transformation-Based Defense applies harmless transformations to incoming samples to neutralize adversarial perturbations without altering their original semantics [23].

**DEST** *TRIM algorithm*. It iteratively removes outlier data points that are not representative of the overall distribution, countering poisoning attacks to restore reliable model performance [3].

**DESO** *System Hardening*. It protects CPS against hardware-level attacks that compromise software and middleware layers, potentially altering model behavior or data integrity [3].

**DEST** *Randomization & Access Controls*. They combine data-space randomization, which alters the representation of data objects to block malicious injections, with hardware access controls. These controls include both physical and logical safeguards to ensure that only authorized entities can access sensitive data and resources [22].

**DESO** *Threat Mitigation Knowledge Integration*. It aims to incorporate structured adversarial techniques and their corresponding defenses into the system’s knowledge base. This enables the system to link detected threats with actionable countermeasures and to strengthen proactive security strategies.

**DEST** *MITRE ATT&CK*. ATT&CK is a taxonomy of tactics, techniques (e.g., Network Sniffing), and mitigations (e.g., Encrypt Sensitive Information). MITRE ATT&CK is a framework that embeds the knowledge base to classify cyberattacks and intrusions [30].

**DESO** *Topology Security Refinement*. It ensures the preservation of security-critical context within system topologies by considering not only physical connections but also higher-level logical and conceptual relationships. This enables more accurate threat modeling and supports effective security verification.

**DEST** *Logical & Conceptual Connections*. They contain two connection types: (i) logical connections (e.g., HTTP, HTTPS, IPSec) that reflect application and network-layer protocols, and (ii) conceptual connections that pair these protocols between component groups. This aims to preserve security-relevant context for verification [30].

**DESO** *Secure Local Network Connection*. It ensures that all communications between sensors, actuators, and local servers occur within a dedicated, isolated network, preventing external access and safeguarding data security [26]. Besides, positioning applications by differentiating geographic locations can foster local data processing and decrease the amount of information sent to cloud infrastructures, thus improving the security of restricted data [7].

**DEST** *Private Intranet*. It connects all device nodes to a local server through switches and routers in an intranet, creating an isolated backbone that protects internal traffic from unauthorized access [18].

**DESO** *Security-Attack Detection*. It seeks to automatically identify and alert on security threats in real time by continuously analyzing system data for deviations from well-known benign and malicious patterns [14].

**DEST *ML-Based Detection*.** It uses machine learning models trained on past attack data to analyze live system activity, and automatically detect abnormal patterns that may signal new threats, enabling real-time security alerts [14].

**DEST *Databus Access Control List(ACL)*** (under **DESO *End-to-End Secure Communication***, see Section A.1). It restricts which entities are allowed to publish data to, or subscribe to data from, the communication databus. By explicitly controlling the rights of publishers and subscribers, this technique prevents unauthorized components from injecting, accessing, or consuming data streams. It complements transport-level protection mechanisms such as TLS by enforcing authorization rules at the data-bus interaction level [8].

**DESO *Runtime Security*.** It aims to continuously monitor the security state of deployed nodes and applications and to trigger reconfiguration actions when security violations are detected. This objective combines runtime threat detection with resource-management decisions, ensuring that applications with security constraints are not kept running on nodes considered insecure [31].

**DEST *Host-Based Monitoring*.** It uses a centralized host-based monitoring architecture to collect and evaluate security audit results from distributed nodes. In the ELASTIC architecture [31], this mechanism relies on Wazuh, a host-based intrusion detection system, together with distributed agents deployed on monitored devices. The centralized monitor controls security parameters for the rest of the nodes, while the agents provide audit results. This design avoids relying on potentially compromised end devices to self-report their security status and enables the system to determine whether a node is secure or threatened.

**DEST *GRM Reallocation*.** It is a runtime adaptation mechanism that records node security status, notifies the Global Resource Manager (GRM), and supports the relocation of security-sensitive applications away from insecure nodes. Security information is stored in the dataClay configuration, while the GRM uses this information to recommend changes in application placement. When a node is detected as insecure, security constraints override other resource-allocation criteria, and affected applications are moved to safer available nodes [31].

**DESO *Holistic Security Modeling*.** It aims to model and analyze security threats by considering technical, social, behavioral, and standards-based aspects together. This goal enables the representation of vulnerabilities, attacks, human factors, security objectives, countermeasures, and concepts derived from standards within a single integrated security model. It is particularly relevant for autonomous vehicle systems, where both technical attacks and social engineering can affect system security [13].

**DEST *Security Abstraction Model (SAM)*.** It is a metamodel for representing attacks, vulnerabilities, security goals, countermeasures, and security concepts derived from standards. SAM can be integrated with system architecture concepts, such as modeled system items, while also incorporating automotive cybersecurity standards. The metamodel extension includes elements for representing social-engineering-related factors, such as human actors, persuasion strategies, resilience, and social-engineering scoring. These elements allow psy-

chological and behavioral vulnerabilities to be analyzed together with technical vulnerabilities [13].

**DESO Risk Assurance.** It aims to evaluate the security risk posture of a distributed system by combining risk management with assurance evidence. Instead of assuming that deployed countermeasures work at their expected strength, this objective evaluates whether mechanisms actually support the intended non-functional properties and adjusts residual risk accordingly. It supports evidence-based assessment of confidentiality, integrity, availability, and other security-relevant properties [1].

**DEST Flow Network.** It models security risk mitigation as a flow network where assets, security mechanisms, and non-functional properties are represented as nodes and arcs. The maximum flow represents the amount of security risk that the deployed mechanisms are able to mitigate, while the difference between total risk and mitigated risk represents residual security risk [1].

**DEST Assurance Model.** It evaluates whether a security mechanism actually supports the non-functional property it is expected to protect. The model links a property, a mitigation mechanism, and a set of test cases used to verify the mechanism’s behavior. The assurance result is then used to adjust the mitigation degree of the mechanism, preventing the system from overestimating the effectiveness of weak or failing countermeasures [1].

**DESO Data-Centric Modeling.** It aims to represent critical system requirements by modeling data items, data dependencies, transformations, communication paths, timing constraints, persistence needs, and protection constraints at design time. It focuses on the security implications of how data are generated, transformed, communicated, stored, and used for actuation.

**DEST SmartData.** It is a data construct enriched with metadata that describes properties such as data origin, semantics, timing, periodicity, security, persistence, and relationships with other data. Security-relevant data can be marked through stereotypes such as secure or persistent, allowing protection needs, protocol overhead, storage requirements, and forensic or monitoring needs to be represented at design time. SmartData helps make protection requirements explicit at the level of the data being sensed, transformed, communicated, stored, or used for actuation [19].

**DEST SmartData Graph.** It represents SmartData elements and their dependencies as a directed graph. The graph captures relationships between data items together with timing requirements, security annotations, persistence annotations, and criticality information. It helps identify critical data paths, protected communication needs, and data dependencies whose compromise may affect confidentiality, integrity, availability, or safe system operation [19].

**DEST Trusted Execution Environment.** (under **DESO Execution Isolation**, see Section A.1) It provides an isolated execution environment for critical network handling outside the control of compromised normal-world software. In the proposed architecture, the network peripheral is assigned to the trusted environment, where a minimal network driver receives packets before they can be affected by the compromised kernel. The trusted environment also separates pack-

ets intended for critical applications from ordinary traffic. As a technique under Execution Isolation, it strengthens Security by protecting critical message delivery from denial, interception, or manipulation by compromised system software [33].

**DEST Trusted I/O.** It gives the trusted environment exclusive, on-demand control over selected input and output peripherals, such as the display and buttons. This allows critical messages to be shown to the user and confirmations to be collected without interference from compromised normal-world software. The mechanism can also be combined with authentication-bound cryptographic operations, where signing or confirmation actions are allowed only after successful user authentication. As a technique under Execution Isolation, it strengthens Security by isolating the user-interaction path from spoofing, blocking, or manipulation [33].

### A.3 Description of Design Objectives and Techniques for Transparency

*Transparency* in CPS ensures that stakeholders can understand how system decisions are made and how these decisions affect system behavior. This is helpful for diagnosing faults, validating safety properties, and building users' trust. To support these needs, we extract and structure 6 design objectives and 5 design techniques, which are reported in Figure 5.

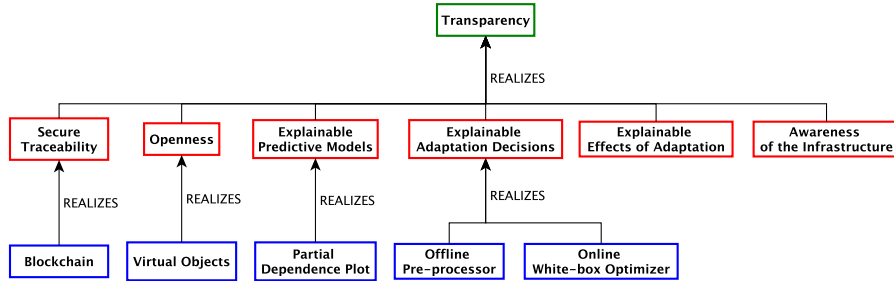


Fig. 5: Knowledge Graph mapping *Transparency* to its design objectives and their associated design techniques.

**DESO Openness.** It provides controlled access to system internals through well-defined interfaces, allowing agents and external services to observe and interact with core components. This improves transparency by making the system's functionality and state visible, understandable, and easier to monitor [32].

**DEST Virtual Objects.** They are deployed above the physical sensors and actuators in agent-based frameworks, and such objects expose standardized Application Programming Interfaces (APIs) [6,15]. These APIs provide transparent

access to the underlying sensors and actuators, revealing the features and capabilities of physical objects. This empowers agents that can establish direct connections with devices via APIs, eliminating the need for proprietary drivers or communication protocols [7].

**DESO** *Explainable Predictive Models*. They provide transparent insights into how predictive algorithms make decisions by providing global explanations [28].

**DEST** *Partial Dependence Plot (PDP)*. It provides a global, model-agnostic view of each feature’s effect on predictions [28].

**DESO** *Explainable Adaptation Decisions*. They aim to make the system decision-making transparent by providing clear justifications for each adaptation step. This helps users understand why certain actions were taken and builds trust in the system’s behavior. Consequently, such decisions support debugging, validation, and compliance with accountability requirements.

**DEST** *Offline Pre-processor*. It takes all classifiers as input and employs global interpretable methods (such as PDPs) to produce explanations of how feature distributions drive requirement satisfaction in the semantic space [28].

**DEST** *Online White-box Optimizer*. This technique is triggered when the system predicts a requirement will not be met. It utilizes prior explanations, such as PDPs and score-PDPs, to adjust controllable features in real-time, guiding the classifier to return “true” with a probability above a set threshold [28].

**DESO** *Explainable Effects of Adaptation*. They ensure visibility into how adaptation actions impact the surrounding environment by clearly exposing the potential consequences of each decision, enabling better understanding, monitoring, and control [28].

**DESO** *Awareness of the Infrastructure*. It ensures that the fog infrastructure is transparent to all users by exposing its configuration, status, and capabilities through standardized interfaces [7].

**DESO** *Secure Traceability*. It aims to make data exchanges, transactions, identities, or records both inspectable and resistant to tampering.

**DEST** *Blockchain*. It provides a decentralized, tamper-resistant ledger for recording transactions, identities, or data exchanges. Its cryptographic mechanisms support secure peer-to-peer transactions, while its traceability helps verify the authenticity of data and transactions. Smart contracts can be treated as part of this technique when they automate and enforce agreements on top of the ledger [16].

## References

1. Anisetti, M., Ardagna, C.A., Bena, N., Foppiani, A.: An assurance-based risk management framework for distributed systems. In: International Conference on Web Services. pp. 482–492. IEEE (2021)
2. Bhagoji, A.N., Chakraborty, S., Mittal, P., Calo, S.: Analyzing federated learning through an adversarial lens. In: International conference on machine learning. pp. 634–643 (2019)

3. Chen, H., Babar, M.A.: Security for machine learning-based software systems: A survey of threats, practices, and challenges. *ACM Computing Surveys* **56**(6), 1–38 (2024)
4. Chen, H., Fu, C., Zhao, J., Koushanfar, F.: Deepinspect: A black-box trojan detection and mitigation framework for deep neural networks. In: *IJCAI*. vol. 2, p. 8 (2019)
5. Choquette-Choo, C.A., Tramer, F., Carlini, N., Papernot, N.: Label-only membership inference attacks. In: *International conference on machine learning*. pp. 1964–1974. PMLR (2021)
6. Cicirelli, F., Guerrieri, A., Spezzano, G., Vinci, A.: An edge-based platform for dynamic smart city applications. *Future Generation Computer Systems* **76**, 106–118 (2017)
7. Da Silva, T.P., Batista, T., Lopes, F., Neto, A.R., Delicato, F.C., Pires, P.F., Da Rocha, A.R.: Fog computing platforms for smart city applications: A survey. *ACM Transactions on Internet Technology* **22**(4), 1–32 (2022)
8. Dintén, R., Martínez, P.L., Zorrilla, M.: Model-based tool for the design, configuration and deployment of data-intensive applications in hybrid environments: An industry 4.0 case study. *Journal of Industrial Information Integration* **41**, 100668 (2024)
9. Dwork, C., Roth, A.: The Algorithmic Foundations of Differential Privacy. *Foundations and Trends in Theoretical Computer Science* **9**(3–4), 211–407 (2014)
10. Enireddy, V., Somasundaram, K., Prabhu, M.R., Babu, D.V., et al.: Data obfuscation technique in cloud security. In: *International Conference on Smart Electronics and Communication (ICOSEC)*. pp. 358–362 (2021)
11. European Parliamentary Research Service: Scientific Foresight - Ethical Aspects of Cyber-Physical Systems. Link - Last Access: 20 March 2026 (2016)
12. Fang, M., Cao, X., Jia, J., Gong, N.: Local model poisoning attacks to {Byzantine-Robust} federated learning. In: *USENIX Security Symposium*. pp. 1605–1622 (2020)
13. Fischer, A., Kolagari, R.T.: Towards robust autonomous vehicle software: Extending sam for a holistic security analysis. In: *International Conference on System Reliability and Safety (ICSRS)*. pp. 578–586. IEEE (2024)
14. Freitas, L., Lelli, V.: Using machine learning on testing iot applications: A systematic mapping. In: *Symposium on Multimedia and the Web*. pp. 348–358 (2022)
15. Giordano, A., Spezzano, G., Vinci, A.: Smart agents and fog computing for smart city applications. In: *Smart Cities: First International Conference (Smart-CT)*. pp. 137–146 (2016)
16. Goumopoulos, C.: Smart city middleware: A survey and a conceptual framework. *IEEE Access* **12**, 4015–4047 (2024)
17. Guo, W., Wang, L., Xing, X., Du, M., Song, D.: Tabor: A highly accurate approach to inspecting and restoring trojan backdoors in ai systems. *arXiv preprint arXiv:1908.01763* (2019)
18. Helal, R., Seghiri, A., Belala, F., Hameurlain, N.: Towards a Formal Modeling Approach for Cyber-Physical Systems Requirements. In: *International Conference on Software and Computer Applications*. pp. 298–309 (2024)
19. Hoffmann, J.L.C., Fröhlich, A.A.: Smartdata: Towards the data-driven design of critical systems. *IEEE Access* (2025)
20. Hutiri, W., Ding, A.Y., Kawsar, F., Mathur, A.: Tiny, always-on, and fragile: Bias propagation through design choices in on-device machine learning workflows. *ACM Transactions on Software Engineering and Methodology* **32**(6), 1–37 (2023)

21. Juuti, M., Szyller, S., Marchal, S., Asokan, N.: Prada: protecting against dnn model stealing attacks. In: European Symposium on Security and Privacy (EuroS&P). pp. 512–527 (2019)
22. Kim, A.A.H.: The impact of platform vulnerabilities in AI systems. Ph.D. thesis, Massachusetts Institute of Technology (2020)
23. Li, Y., Zhai, T., Wu, B., Jiang, Y., Li, Z., Xia, S.: Rethinking the trigger of backdoor attack. arXiv preprint arXiv:2004.04692 (2020)
24. Liu, K., Dolan-Gavitt, B., Garg, S.: Fine-pruning: Defending against backdoor-ing attacks on deep neural networks. In: International symposium on research in attacks, intrusions, and defenses. pp. 273–294 (2018)
25. Lo, S.K., Lu, Q., Zhu, L., Paik, H.Y., Xu, X., Wang, C.: Architectural Patterns for the Design of Federated Learning Systems. *Journal of Systems and Software* **191**, 111357 (2022)
26. Maruf, M.A., Azim, A.: Automated features and requirements identification for improving cps software reuse using topic modeling. In: Proceedings of the International Conference on Cyber-Physical Systems (with CPS-IoT). pp. 262–263 (2023)
27. Milossi, M., Alexandropoulou-Egyptiadou, E., Psannis, K.E.: AI ethics: algorithmic determinism or self-determination? The GDPR approach. *Ieee Access* **9**, 58455–58466 (2021)
28. Negri, F.R., Nicolosi, N., Camilli, M., Mirandola, R.: Explanation-driven self-adaptation using model-agnostic interpretable machine learning. In: Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems. pp. 189–199 (2024)
29. Nguyen, G., Devroey, X.: A3s3-automated android audit of safety and security signals. In: International Conference on Advanced Information Systems Engineering. pp. 205–212. Springer (2025)
30. Ooi, S.E., Beuran, R., Tan, Y., Kuroda, T., Kuwahara, T., Fujita, N.: Secureweaver: Intent-driven secure system designer. In: Proceedings of the 2022 ACM Workshop on Secure and Trustworthy Cyber-Physical Systems. pp. 107–116 (2022)
31. Sousa, R., Pinho, L.M., Barros, A., Gonzalez-Hierro, M., Zubia, C., Sabate, E., Kartsakli, E.: Managing non-functional requirements in an elastic edge-cloud continuum. *Ada User J* **43**(4), 245–248 (2022)
32. Torresen, J., Saplacan, D., Baselizadeh, A., Mahler, T.: Machine excellence trade-offs to ethical and legal perspectives. In: Conference on Artificial Intelligence (CAI). pp. 237–240 (2023)
33. Van Eyck, T., More, S., Draschbacher, F., Michiels, S., Hughes, D.: Don’t stop receiving: Ensuring availability of critical services on compromised mobile devices. In: International Symposium on Network Computing and Applications. pp. 131–140. IEEE (2025)
34. Verma, R., Dhanda, N., Nagar, V.: Enhancing security with in-depth analysis of brute-force attack on secure hashing algorithms. In: Proceedings of Trends in Electronics and Health Informatics: TEHI 2021, pp. 513–522. Springer (2022)
35. Wang, Y., Mianjy, P., Arora, R.: Robust learning for data poisoning attacks. In: International Conference on Machine Learning. pp. 10859–10869 (2021)
36. Xiang, Z., Miller, D.J., Kesidis, G.: Detection of backdoors in trained classifiers without access to the training set. *IEEE Transactions on Neural Networks and Learning Systems* **33**(3), 1177–1191 (2020)
37. Yuntao, L., Yang, X., Ankur, S.: Neural trojans. In: Proceedings of the IEEE International Conference on Computer Design (ICCD). pp. 45–48 (2017)
38. Zhu, Y., Cheng, Y., Zhou, H., Lu, Y.: Hermes attack: Steal DNN models with lossless inference accuracy. In: USENIX Security Symposium (2021)